



PDF Download
3576914.3587493.pdf
07 March 2026
Total Citations: 5
Total Downloads: 620

Latest updates: <https://dl.acm.org/doi/10.1145/3576914.3587493>

RESEARCH-ARTICLE

Adversarial Attacks on Adaptive Cruise Control Systems

YANAN GUO, University of Pittsburgh, Pittsburgh, PA, United States

TAKAMI SATO, University of California, Irvine, Irvine, CA, United States

YULONG CAO, University of Michigan, Ann Arbor, Ann Arbor, MI, United States

QI ALFRED CHEN, University of California, Irvine, Irvine, CA, United States

YUEQIANG CHENG

Open Access Support provided by:

University of Michigan, Ann Arbor

University of California, Irvine

University of Pittsburgh

Published: 09 May 2023

Citation in BibTeX format

CPS-IoT Week '23: Cyber-Physical
Systems and Internet of Things Week
2023

May 9 - 12, 2023

TX, San Antonio, USA

Conference Sponsors:
SIGBED

Adversarial Attacks on Adaptive Cruise Control Systems

Yanan Guo
University of Pittsburgh
Pittsburgh, PA, USA
yag45@pitt.edu

Takami Sato
University of California, Irvine
Irvine, CA, USA
takamis@uci.edu

Yulong Cao
University of Michigan
Ann Arbor, MI, USA
yulongc@umich.edu

Qi Alfred Chen
University of California, Irvine
Irvine, CA, USA
alfchen@uci.edu

Yueqiang Cheng
NIO
San Jose, CA, USA
strongerwill@gmail.com

ABSTRACT

DNN-based Adaptive Cruise Control (ACC) systems are very convenient but also safety critical. Although prior work has explored physical adversarial attacks on **DNN models**, those attacks are mostly static and their effects on a real-world ACC system are not clear. In this work, we propose the first end-to-end attack on ACC systems, and we test the safety indication on the state-of-the-art ACC products. The experimental results show that our approach can make the vehicle driving with ACC accelerate unsafely and cause a rear-end collision.

CCS CONCEPTS

• Security and privacy → Software security engineering.

KEYWORDS

autonomous driving, **adversarial attack**

ACM Reference Format:

Yanan Guo, Takami Sato, Yulong Cao, Qi Alfred Chen, and Yueqiang Cheng. 2023. Adversarial Attacks on Adaptive Cruise Control Systems. In *Cyber-Physical Systems and Internet of Things Week 2023 (CPS-IoT Week Workshops '23)*, May 09–12, 2023, San Antonio, TX, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3576914.3587493>

1 INTRODUCTION

Adaptive Cruise Control (ACC) is a Level-2 driving automation technology that can automatically adjust the vehicle's velocity based on the velocity and position of the vehicle in front (a.k.a. the lead), in order to keep pace with the lead, as well as staying at a safe distance from the lead. Due to its high convenience for human drivers, ACC has been available in vehicles from many manufacturers such as Honda, Toyota, Nissan, Chevrolet, and Volkswagen. Although convenient, ACC is extremely safety critical since if it makes the wrong driving decision and accelerates/brakes unsafely, it may rear-end the vehicle in front or get rear-ended by the vehicle behind.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CPS-IoT Week Workshops '23, May 09–12, 2023, San Antonio, TX, USA

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0049-1/23/05...\$15.00

<https://doi.org/10.1145/3576914.3587493>

The most critical step in ACC is lead detection. To achieve this, most ACC systems use camera as the only or major sensor to perceive the driving environment (e.g., [1]). Due to its high accuracy, Deep Neural Networks (DNNs) are widely adopted in ACC systems to extract the status of the lead from the image captured by camera. However, many previous works have shown that DNN models are vulnerable to deliberately crafted inputs (a.k.a. adversarial examples). In fact, adding small but intentionally selected perturbations to an image can make a DNN model give wrong outputs/decisions in many domains, such as image classification [12] and object detection[28].

However, early research works (e.g., [8]) on generating adversarial examples mainly focus on the *digital field*, where it is assumed that the adversary can directly manipulate the pixels of the input images or videos of a DNN model. Such assumption is not practical when attacking a real-world DNN system such as the ACC system because the adversary needs to have access to the internal data of the ACC system in order to modify the input of the DNN model. Instead, a more realistic scenario for adversarial attacks is that the adversary modifies a physical object which will affect the images taken by the camera in the DNN system and thus will affect the inputs of the DNN model.

More recently, many works have proposed adversarial examples that can be printed and attached to real-world objects, making some DNN models used in Autonomous Driving (AD) systems malfunction. For example, in [25], a real stop sign with an adversarial poster or sticker on it cannot be detected by object detection models such as YOLO v2 [19]. Another work [31] also targets stop signs and can **make the attack more robust using their new attack methodologies such as nested adversarial examples**. In [29], after applying a mask on a license plate, this license plate can no longer be recognized by an SSD model. In these works, the proposed adversarial examples were tested under various angles, distances, and illumination conditions. However, there are still two common limitations of these attacks.

First, these attacks are all *static* attacks that have only been evaluated with static images and have not been tested with a production-grade driving system. We argue that it is unclear how the proposed adversarial attacks affect a real-world driving system without evaluating it with an end-to-end setup. For example, the output of a DNN model is not used to control the vehicle directly. Instead, they are usually used in a Model Predictive Control (MPC) system to calculate some parameters which will be further used in the vehicle control (e.g., the control of the gas and brake). Second, these

previous attacks mainly focus on attacking 2D object detection models. In fact, 3D object detection models are much more popular in real-life driving systems [3, 4]. It is not clear whether their attack methods can be directly used in such driving systems.

In a very recent work [22], an end-to-end attack on lane-keeping assistance system is proposed; however, this approach does not apply to ACC systems. In this work, we design and implement the first approach to attack DNN-based ACC systems. To achieve this, we study the fusion and control details of ACC systems and identify the key requirements for causing a rear-end crash on a vehicle driving with ACC. Then, we propose a new objective function targeting both the confidence score and the detected distance of the leading vehicle. To show the safety implications, we test our attack on multiple real-life ACC systems in driving simulators. The experimental results show that our attack can cause a rear-end collision between two vehicles. Anonymized videos of this attack are available at <https://sites.google.com/view/acc-adv/>.

2 RELATED WORK

Physical Adversarial Attacks. Digital attacks where the adversary directly modifies the pixels of the input images to a DNN model are not practical in many scenarios. In contrast, a more realistic threat model for adversarial attacks is that the adversary can only control the objects in the physical world, instead of the pixels of the DNN input, because the latter is usually inaccessible internal data of a DNN-based system. In 2018, Kurakin et al. first showed that adversarial examples will stay effective when they are printed out and retaken by a smartphone camera [15]. Athalye et al. later improved this work and created adversarial examples that are robust against a set of two-dimensional synthetic transformations [7]. Sharif et al. successfully attacked face recognition system [24]: by printing adversarial perturbations on the frame of eyeglasses, the person in the image is no longer recognized or is recognized as a wrong person. Kevin et al. made road signs misclassified by DNN models through placing carefully crafted patches on the signs [12].

As for adversarial attacks related to object detection, Lu et al. tried to attack YOLO object detector in 2017 [18]. Their experiments were done by printing adversarial road signs on papers. Chen et al. attacked the Faster R-CNN object detector by replacing the road signs with adversarial poster perturbations [9]. At the same time, Kevin et al. attacked Faster R-CNN and YOLO v2 object detectors through similar techniques [25]. Later in 2019, Yue et al. proposed new attack methods including Feature-interference Reinforcement and Enhanced Realistic Constraints which can make the attack on road signs more robust against physical variants including the weather, illumination, and distance [31]. In addition, Kaichen et al. first proposed an adversarial attack on SSD model for license plates. This attack can also be transferred to many object detection models [29].

Defenses against Adversarial Attacks. Florian et al. proposed adversarial training [26] which can make the DNN model itself more robust in the training stage. Adversarial training is much slower than normal DNN training; thus, later [23] was proposed which can significantly reduce the training time. In addition, many approaches have been proposed to defend localized patch based adversarial attacks. Muzammal created Local Gradient Smoothing

which can weaken the effect of an adversarial patch by adding noise on it [16]. Jamie et al. proposed to remove the adversarial patch using similar methods with removing watermarks [14]. Edward et al. designed SentiNet [10] which can detect an adversarial patch using the feature that localized universal attacks are robust against a variety of artifacts. However, these defenses are all empirical defenses. Recently, many certifiable defenses on patch attacks have been proposed [17, 27]. Though effective, these defenses often require re-training the DNN model.

3 BACKGROUND

ACC is one of the key functions in modern Level-2 AD systems. With ACC, the driver only needs to set the maximum velocity V_{MAX} , (a.k.a. the cruise velocity), and the ACC system automatically adjusts the vehicle velocity within $[0, V_{MAX}]$: if there is a close vehicle (a.k.a. a lead) in front, the system controls the velocity of the ego vehicle based on the position and velocity of the lead. For example, if a slower vehicle moves in front of the ego vehicle, the ACC system controls the ego vehicle to slow down; when the lead accelerates and the velocity is higher than V_{MAX} or when the lead moves out of the way, the ego vehicle accelerates to achieve V_{MAX} .

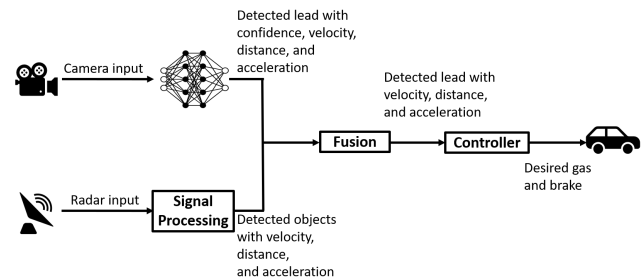


Figure 1: The overview of the ACC system in OpenPilot.

We use the ACC system design in OpenPilot [4] to introduce the details. As shown in Figure 1, this ACC system operates in three steps:

Lead Detection (LD). Lead detection is the most critical step in an ACC system, since the driving decisions are made mainly based on the sensor inputs. Radars are very accurate at detecting and tracking the status of objects (e.g., velocity), but they cannot tell if the detected object is a lead (vehicle). Thus, in OpenPilot and many other recent ACC products (e.g., [1, 2]), both camera and radar sensors are used to perceive the environment. First, as shown in Figure 1, a DNN model is used to extract the lead information from each camera frame¹. In frame t , the detected lead is associated with the confidence score $s_t^c \in [0, 1]$, and the status of the lead $y_t^c = (d_t^c, l_t^c, v_t^c, a_t^c)$, where d_t^c and l_t^c are the distances between the lead and ego vehicle along the driving direction and lateral direction; v_t^c and a_t^c are the velocity and acceleration of the lead (relative to the ego vehicle) along the driving direction; we use superscript c, r for camera and radar (mentioned later). Second, traditional signal processing is used to extract the information of

¹Though rare, OpenPilot can detect multiple leads when there are multiple vehicles in front in the same lane with the ego vehicle. Here we only consider the case in which one lead is detected since other leads are usually obstructed by the closest lead.



Figure 2: Trucks with advertisement on the side and back.

the objects detected by radar. The i -th object in frame t can be represented as $y_t^{r,i} = (d_t^{r,i}, l_t^{r,i}, v_t^{r,i}, a_t^{r,i})$.

Fusion. The detection results from camera and radar are fused to get the final result. The detailed fusion process can be found in [4]. In short, 1) if $s_t < 0.5$, this detected lead is treated invalid; 2) if $s_t > 0.5$, y_t^c is compared with each $y_t^{r,i}$ in frame t to find the best match.

Longitudinal control and vehicle actuation. In each frame t , the fused status of a valid lead ($y_t^* = (d_t^*, l_t^*, v_t^*, a_t^*)$) is sent to a **Model Predictive Control (MPC)** [20] module to calculate the desired velocity and acceleration for the next step. Then, the desired velocity and acceleration are used by a **Proportional-Integral (PI) control loop** [13] to control the gas and brake of the vehicle. With a detected lead, the control objective is to smoothly achieve the same velocity with the lead as well as maintaining the safe distance with the lead. The safe distance is defined as:

$$d_t^{safe} = v_t^{ego} * TR - (v_t^{lead} - v_t^{ego}) * TR + v_t^{ego} * v_t^{ego} / (2G) - v_t^{lead} * v_t^{lead} / (2G) + 4. \quad (1)$$

The safe distance in frame t , d_t^{safe} , is related to the absolute velocity of the lead and ego vehicle (v_t^{lead} and v_t^{ego}). It is the distance the ego vehicle needs if the lead starts braking at $G \text{ m/s}^2$ and TR seconds later the ego vehicle starts braking too at $G \text{ m/s}^2$ and is able to eventually stop at 4 m away from the lead. In OpenPilot v0.8.3, $TR = 1.8$ and $G = 9.81$.

4 ADVERSARIAL ATTACK ON ACC

4.1 Threat Model and Problem Formulation

We assume an attacker who aims to attack the ACC system on the victim vehicle to cause a rear-end collision between the victim vehicle and the lead in front of it. The attacker can achieve this by manipulating objects in the physical world to “fool” the DNN model in the ACC system. We assume that the attacker can possess the same ACC system as the one on the victim vehicle and has full knowledge of the ACC algorithm by either reverse-engineering the algorithm or accessing the code repository if it is open-source.

Physical approach. We focus on the scenario in which the attacker tries to launch the attack by placing an adversarial patch on the back of a vehicle (i.e. an adversarial vehicle), and cause the vehicle behind which is driving with ACC to rear-end the adversarial vehicle. We choose this attack scenario for two reasons:

- R1: **Feasibility.** In many ACC systems, a **Region of Interest (ROI)** filtering is performed to the raw camera frame before the DNN model is applied (Figure 3). This is to extract the most import

area to improve the accuracy of lead detection [30]. Thus, modifying objects out of the ROI area such as the surrounding buildings will not affect the lead detection.

- R2: **Legality.** Within the ROI area, some previous works proposed to manipulate the license plate of the lead [29] or the patches on the road [22] to achieve the attack. However, such approaches can be illegal in many cities/countries. **In contrast, modifying the appearance of a vehicle is more likely legitimate:** 1) if the attacker owns the adversarial vehicle, then he is allowed to change the vehicle appearance (without covering the light, license plate, etc.) according to the law in many places; 2) the attacker can also use a vehicle that has advertisement on the back as the adversarial vehicle. As shown in Figure 2, many truck companies offer the back of the truck as a place for advertisement. Thus, the attacker can control the back appearance of a truck legally by advertising.

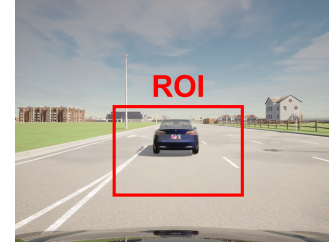


Figure 3: The ROI area of an input image.

Attack incentives. We consider two possible incentives: *business competition* and *automotive insurance fraud*. First, one ACC system provider may build an attack to cause an accident on a vehicle driving with a rival company’s ACC system, and thus damage its reputation as well as gaining unfair competitive advantages. Second, automotive insurance frauds happen very frequently in many countries [6]. Adversarial attacks can be a new way to perform the fraud: the attacker can get compensation from the insurance company if the vehicle driving with ACC behind him is affected by the patch and rear-ends the attacker’s vehicle.

4.2 Attack Algorithm

Our goal is to make a vehicle driving with an ACC system (ACC vehicle for short) accelerate unsafely and eventually cause a rear-end collision. As shown in Section 3, when driving at a velocity less than V_{MAX} in frame t , **an ACC vehicle will accelerate when one of the two following cases occurs: 1) $s_t < 0.5$; 2) $s_t > 0.5$, $d_t^c > d_t^{safe} + \lambda$.** λ is caused by the fusion process and is about $0.25 * d_t^c$. Thus, different than previous attacks on object detection models that only focus on reducing s_t to cause a disappearance attack, we include both s_t and d_t^c in our objective to learn adversarial examples. And later in Section 5 we will show that this change is very important to the attack.

Sequential attack. To have end-to-end impact on an ACC system, the attack needs to affect a sufficient number of consecutive camera frames. In fact, the attacks on later frames depend on the ones on earlier frames. For example, if the adversarial patch successfully causes the lead to be not detected or to be considered far away in frame t , then the ACC system will control the vehicle to

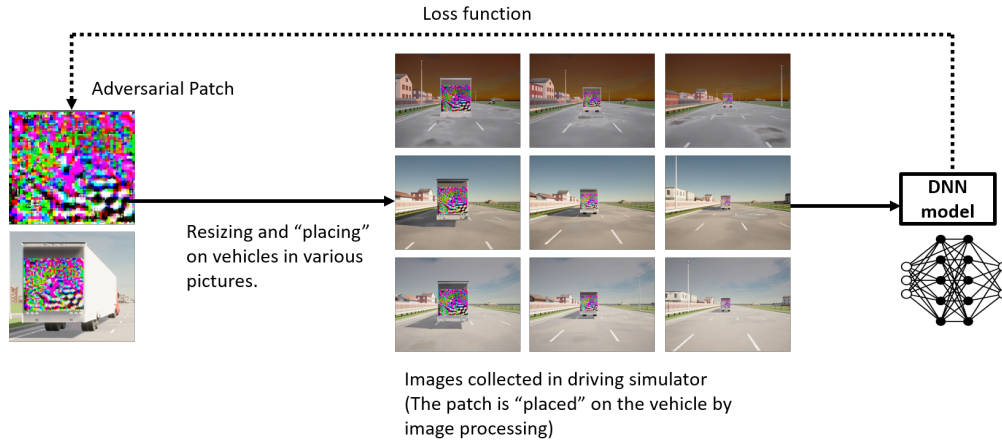


Figure 4: The work flow of adversarial patch generation.

accelerate, and the patch (and also the lead) will become larger in the image captured in frame $t+1$. Thus, to cause a collision on an ACC vehicle which is following a lead with a following distance D_0 (i.e. the distance between the lead and the ACC vehicle), we need an adversarial patch that is robust when the following distance lies in $[0, D_0]$. To achieve this, we apply Expectation over Transform (EOT) [7] in our attack.

Specifically, given the digital template of the patch on the lead x (e.g., the advertisement patch), we add perturbation δ on it to generate the adversarial patch $x' = x + \delta$. Then, to apply EOT, we simulate the images taken by the camera under different conditions: we collect real input images from a driving simulator which are taken with the following distance lying in $[0, D_0]$, under different weathers and backgrounds, and rescale the adversarial patch to place it on the vehicles in the sampled images, as shown in Figure 4. The collected images can be denoted as

$$\begin{aligned} x' &= x + \delta \\ z_i &= T(x', f_i) \end{aligned} \quad (2)$$

where T denotes the overall process of rescaling and placing the adversarial patch, f_i is the i -th sampled image.

4.3 Objective

The overall loss function is a combination of the adversarial loss and the Total Variation (TV) loss:

$$\mathcal{L} = \underbrace{\mathcal{L}_{conf} + \lambda_1 \mathcal{L}_{dis}}_{\mathcal{L}_{adv}} + \lambda_2 \mathcal{L}_{TV}. \quad (3)$$

We consider the following mathematical formulation for the adversarial loss:

$$\begin{aligned} \mathcal{L}_{adv} &= \mathcal{L}_{conf} + \lambda_1 \mathcal{L}_{dis} \\ &= \mathbb{E}_{z_i \in Z} -\log(1 - s(z_i)) - \lambda_1 |d(z_i) / d^f(f_i)| \end{aligned} \quad (4)$$

where $s(z_i)$ and $d(z_i)$ denote the confidence score of the detected lead and the detected distance between the lead and the ego vehicle, with the input image z_i ; $d^f(f_i)$ denotes the following distance when the image f_i was taken. Z is the set of simulated images. We use binary cross entropy to minimize the confidence score of

the detected lead, and we add the detected distance in the loss to maximize it.

In addition to the adversarial loss, we use TV loss as done in [25], to smooth the perturbation pattern and make it continuous, in case that the printing machine cannot accurately distinguish pixelated patterns. The TV loss of the adversarial patch can be formulated as:

$$\mathcal{L}_{TV} = \sum_{i,j} |\delta_{i+1,j} - \delta_{i,j}| + |\delta_{i,j+1} - \delta_{i,j}| \quad (5)$$



Figure 5: The adversarial patch used in the experiments.

5 EVALUATION

We evaluate our attack method in this section. First, we apply the attack to real-life ACC systems and test the end-to-end effect of the attack in driving simulators. Then, we print out the adversarial patch and place it on a vehicle model to test the attack in the physical world.

5.1 End-to-End Effect of the Attack

Evaluation on OpenPilot. We first evaluate our method on the state-of-the-art open-source Level-2 AD system, OpenPilot [4], since it has been shown to have better performance than products from many other manufacturers [5]. The ACC system of OpenPilot is introduced in Section 3. To test the end-to-end security/safety effect of our attack, we perform the evaluation in a driving simulator named CARLA [11]. Specifically, we first “drive” a vehicle (controlled by OpenPilot) in CARLA following a lead (a box truck) to collect some input images (Step 1). Then, based on these images, we learn the adversarial patch following the methodology in Figure 4 (Step 2). Next, we modify the vehicle appearance in CARLA to place the adversarial patch on the back of the lead (Step 3). Finally,

$\mathcal{L}_{conf} = E[-\log(1 - s(z_i))]$: binary cross entropy per minimizzare il confidence score.

Quando $s(z_i)$ si avvicina a 0, $-\log(1-s)$ si avvicina a 0 (loss minima). Quando $s(z_i)$ è vicino a 1, la loss esplode. Quindi minimizzare questa loss spinge s verso 0.

$\mathcal{L}_{dis} = E[-|d(z_i) / d^f(f_i)|]$: il rapporto tra la distanza percepita dal detector e la distanza reale

di inseguimento, con segno negativo perché si vuole massimizzare la distanza percepita. Si normalizza per la distanza reale $d^f(f_i)$, il che bilancia l'importanza di questo termine a diverse distanze.

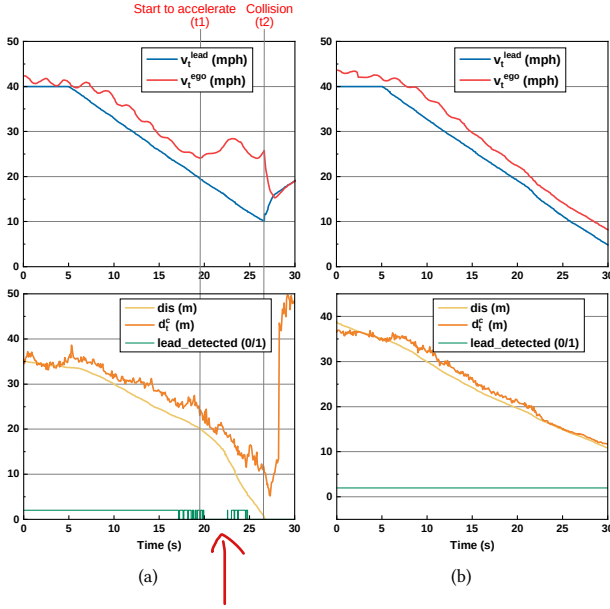


Figure 6: The detailed process of the attack (a) and benign case (b), where dis denotes the following distance, $dis_detected$ is a binary which represents if $s_t > 0.5$ is true.

we “drive” the OpenPilot vehicle again to follow the lead with the adversarial patch on its back to test the attack (Step 4).

In Step 4 above, we focus on a scenario where the lead is slowing down. There are *three phases* in this scenario:

- P1: The ACC vehicle is following the lead as normal at a constant velocity which is 30-80 mph, and the following distance is thus 28-70 m.
- P2: The lead starts to slow down due to some special situation in front of it. Later, the ACC vehicle starts to slow down accordingly, and it will get closer to the lead (because d_t^{safe} is smaller with lower velocity).
- P3: When the distance between the two vehicles is smaller than a threshold (D^*), the adversarial patch starts to take effect and the ACC vehicle starts to accelerate until hitting the lead.

To achieve the above attack, we train the adversarial patch to be robust when the following distance lies in $[0, D^*]$. D^* is 20 m in our experiments. We choose this attack scenario due to its good stealthiness. When the lead is at a high velocity or it just starts to slow down, the driver of the ACC vehicle may pay full attention to ensure that ACC is working correctly. However, after the driver notices that his vehicle is taking the right action (i.e. slowing down accordingly), he will very likely be relaxed/distracted and fully trusting the ACC system. Then, when the vehicle suddenly stops slowing down and starts to accelerate, it may take the driver longer than normal to react and take control.

Results. For the loss function, we set $\lambda_1 = 0.01$, $\lambda_2 = 0.001$. The adversarial patch is shown in Figure 5. Figure 6 shows the details of the ACC vehicle’s behavior when the lead is slowing down from 40 mph, with/without the adversarial patch. Here we assume that the speed limit of the road is 45 mph, i.e. the driver sets V_{MAX} to be 45 mph. At the beginning when $t < 5$ s, the ACC vehicle is

following the lead at ~ 40 mph as normal. When $t = 5$ s, the lead starts slowing down smoothly to eventually stop, and in the benign case, the ACC vehicle will also slow down and eventually stop at ~ 4 m away from the lead. In contrast, in the attack case, although the ACC vehicle slows down at the beginning, when $t = 17$ s and the following distance is 22.5 m, the patch takes effect and the ACC vehicle starts to accelerate at $t_1 = 19.5$ s and crashes into the lead at $t_2 = 26.6$ s. The velocity of the lead increases after t_2 because it is pushed by the ACC vehicle.

As shown in Figure 6, from 22.5 s to 24.5 s the attack on s_t is not very robust due to the variations on illumination, background, and other conditions, and the ACC vehicle thus starts to slow down. However, since we also attack d_t^c and make it much larger than the real following distance, the ACC vehicle does not slow down dramatically although there is a detected lead. d_t^c is not as sensitive to the mentioned variations as s_t because the DNN in OpenPilot is very robust on s_t . We do not only attack d_t^c because the planner limits the acceleration when a lead is detected.

Evaluation on Apollo. We then evaluate our attack on a popular AD system, Apollo [3]. Apollo is a Level-3/Level-4 AD system and thus does not have the ACC function. However, when there is a relatively close vehicle in front of the AD vehicle driving with Apollo, this AD vehicle will adjust the velocity base on the position and velocity of the vehicle in front (i.e. the lead), which is similar with the ACC function. Thus, we also test our attack in this system for generality. We perform the evaluation in the driving simulator LGSVL [21]. Note that here we do not use CARLA due to compatibility issue. But we still use a box truck as the lead and follow the same three steps in attacking OpenPilot.

Results. Instead of causing a rear-end collision when the lead is slowing down. We focus on a different scenario where the lead is starting. Specifically, the lead is about 80-100 m in front of the AD vehicle. At the beginning, both of the vehicles are stopped. When the lead starts, the AD vehicle also starts. In the first experiment, where there is no adversarial attack (as shown in Figure 7 (a)), the AD vehicle will be able to first accelerate to get closer to the lead and then slow down to maintain a safe distance with the lead. However, when we apply an adversarial patch on the lead, (Figure 7 (b)), at the beginning when the lead is still far away, the patch does not have any effect, the AD vehicle accelerates as normal. However, once the AD vehicle gets closer to the lead, the patch starts to take effect, and the AD vehicle driving with Apollo can no longer detect the lead and thus keep accelerating when it is supposed to slow down, until hitting the lead. Note that in this experiment, once the patch starts to take effect, it is very robust when the distance varies, different than the case in OpenPilot. We believe this is because the 3D detection model in Apollo is more vulnerable than the one used in OpenPilot.

In addition, as one can observe in Figure 7, the adversarial patch and the benign patch (i.e. the advertisement) look almost identical to human being’s eyes. This makes the attack very stealthy and thus powerful.

Results. The experimental results show that our attack is always robust against various illumination condition: with a fixed distance, the binary lead detection result always stays the same when changing the illumination condition. In addition, the attack is relatively robust against the distance between the camera and the leading

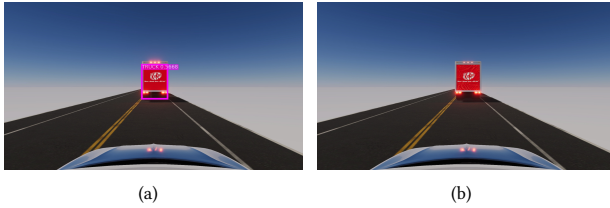


Figure 7: A truck with benign advertisement on the back (a), or adversarial patch on the back (b).

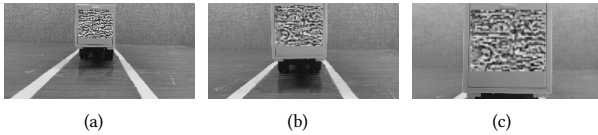


Figure 8: The appearance of a miniature truck with the adversarial patch on the back; showing with different distances between the camera and the truck.

truck. From the 100 taken pictures with a leading truck (with the adversarial patch), OpenPilot cannot detect the truck in 79 of these pictures. Similar with the results shown in Figure 6, the patch does not work well when the truck is too far from the camera.

6 CONCLUSION

In this work, we design and implement the first end-to-end attack on real-world ACC systems. We first propose a realistic threat model for a real-world ACC attack where the attacker places an adversarial patch on the lead to pretend to be mobile advertisement. Under this threat model, we build an ACC attack in which we collect input data by driving the ACC vehicle in an AD simulator and use the collected input data to train a robust adversarial patch. We evaluate our approach on the state-of-the-art real-life ACC systems including OpenPilot and Apollo. The experimental results show that our attack can successfully cause a rear-end collision on a vehicle driving with ACC in AD simulators.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their insightful feedback. We would also like to thank Christopher DiPalma for his constructive contribution to this work. This work is supported in part by NSF CNS-1932464, CNS-1929771, CNS-2145493, and USDOT UTC Grant 69A3552047138.

REFERENCES

- [1] [n.d.]. Intelligent Cruise Control. nissan-global.com/EN/TECHNOLOGY/OVERVIEW/icc.html.
- [2] [n.d.]. Toyota Safety Sense (TSS). <https://www.buyatoyota.com/home/tools/toyota-safety-sense/>.
- [3] 2017. Apollo: Open Source Autonomous Driving. <https://github.com/ApolloAuto/apollo>.
- [4] 2018. OpenPilot: Open Source Driving Agent. <https://github.com/commaai/openpilot>.
- [5] 2018. We hit the road with Comma.ai's assisted-driving tech at CES 2020. <https://www.cnet.com/roadshow/news/comma-ai-assisted-driving-george-hotz-ces-2020/>.
- [6] 2021. 6 Types of car insurance fraud. <https://www.bankrate.com/insurance/car/fraud>.
- [7] Anish Athalye, Logan Engstrom, Andrew Ilyas, and Kevin Kwok. 2018. Synthesizing robust adversarial examples. In *International conference on machine learning*. PMLR, 284–293.
- [8] Nicholas Carlini and David Wagner. 2017. Towards evaluating the robustness of neural networks. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 39–57.
- [9] Shang-Tse Chen, Cory Cornelius, Jason Martin, and Duen Horng Polo Chau. 2018. Shapeshifter: Robust physical adversarial attack on faster r-cnn object detector. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 52–68.
- [10] Edward Chou, Florian Tramèr, Giancarlo Pellegrino, and Dan Boneh. 2018. Sentinet: Detecting physical attacks against deep learning systems. (2018).
- [11] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. 2017. CARLA: An Open Urban Driving Simulator. In *Proceedings of the 1st Annual Conference on Robot Learning*. 1–16.
- [12] Kevin Eykholt, Ivan Evtimov, Earlene Fernandes, Bo Li, Amir Rahmati, Chaowei Xiao, Atul Prakash, Tadayoshi Kohno, and Dawn Song. 2018. Robust physical-world attacks on deep learning visual classification. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 1625–1634.
- [13] Tore Hägglund. 1996. An industrial dead-time compensating PI controller. *Control Engineering Practice* 4, 6 (1996), 749–756.
- [14] Jamie Hayes. 2018. On visible adversarial perturbations & digital watermarking. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*. 1597–1604.
- [15] Alexey Kurakin, Ian Goodfellow, Samy Bengio, et al. 2016. Adversarial examples in the physical world.
- [16] Songhun Kwak, Kwanghun Kim, Kwangnam Choe, and Kumchol Yun. 2020. A local gradient smoothing method for solving strong form governing equation. *European Journal of Mechanics-A/Solids* 84 (2020), 104073.
- [17] Alexander Levine and Soheil Feizi. 2020. (De) Randomized Smoothing for Certified Defense against Patch Attacks. *arXiv preprint arXiv:2002.10733* (2020).
- [18] Jiajun Lu, Hussein Sibai, Evan Fabry, and David Forsyth. 2017. No need to worry about adversarial examples in object detection in autonomous vehicles. *arXiv preprint arXiv:1707.03501* (2017).
- [19] Joseph Redmon and Ali Farhadi. 2017. YOLO9000: better, faster, stronger. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 7263–7271.
- [20] Jacques Richalet, André Rault, JL Testud, and J Papon. 1978. Model predictive heuristic control. *Automatica (journal of IFAC)* 14, 5 (1978), 413–428.
- [21] Guodong Rong, Byung Hyun Shin, Hadi Tabatabaee, Qiang Lu, Steve Lemke, Martin Mozeiko, Eric Boise, Geehoon Uhm, Mark Gerow, Shalin Mehta, et al. 2020. LGSVL Simulator: A High Fidelity Simulator for Autonomous Driving. *arXiv preprint arXiv:2005.03778* (2020).
- [22] Takami Sato, Junjie Shen, Ningfei Wang, Yunhan Jack Jia, Xue Lin, and Qi Alfred Chen. 2020. Hold tight and never let go: Security of deep learning based automated lane centering under physical-world attack. *arXiv preprint arXiv:2009.06701* (2020).
- [23] Ali Shafahi, Mahyar Najibi, Amin Ghiasi, Zheng Xu, John Dickerson, Christoph Studer, Larry S Davis, Gavin Taylor, and Tom Goldstein. 2019. Adversarial training for free! *arXiv preprint arXiv:1904.12843* (2019).
- [24] Mahmood Sharif, Sruti Bhagavatula, Lujo Bauer, and Michael K Reiter. 2016. Accesorize to a crime: Real and stealthy attacks on state-of-the-art face recognition. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 1528–1540.
- [25] Dawn Song, Kevin Eykholt, Ivan Evtimov, Earlene Fernandes, Bo Li, Amir Rahmati, Florian Tramer, Atul Prakash, and Tadayoshi Kohno. 2018. Physical adversarial examples for object detectors. In *12th {USENIX} Workshop on Offensive Technologies ({WOOT})*. 18.
- [26] Florian Tramèr, Alexey Kurakin, Nicolas Papernot, Ian Goodfellow, Dan Boneh, and Patrick McDaniel. 2017. Ensemble adversarial training: Attacks and defenses. *arXiv preprint arXiv:1705.07204* (2017).
- [27] Chong Xiang, Arjun Nitin Bhagoji, Vikash Sehwal, and Prateek Mittal. 2021. Patchguard: A provably robust defense against adversarial patches via small receptive fields and masking. In *30th {USENIX} Security Symposium ({USENIX} Security 21)*.
- [28] Cihang Xie, Jianyu Wang, Zhishuai Zhang, Yuyin Zhou, Lingxi Xie, and Alan Yuille. 2017. Adversarial examples for semantic segmentation and object detection. In *Proceedings of the IEEE International Conference on Computer Vision*. 1369–1378.
- [29] Kaichen Yang, Tzungyu Tsai, Honggang Yu, Tsung-Yi Ho, and Yier Jin. 2020. Beyond Digital Domain: Fooling Deep Learning Based Recognition System in Physical World. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 1088–1095.
- [30] Sibel Yenikaya, Gökhan Yenikaya, and Ekrem Düven. 2013. Keeping the Vehicle on the Road: A Survey on on-Road Lane Detection Systems. *ACM Comput. Surv.* 46, 1, Article 2 (July 2013), 43 pages.
- [31] Yue Zhao, Hong Zhu, Ruigang Liang, Qintao Shen, Shengzhi Zhang, and Kai Chen. 2019. Seeing isn't believing: Towards more robust adversarial attack against real world object detectors. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 1989–2004.